به نام خدا

دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس الگوریتم پیشرفته

تمرین هفتم

نام و نام خانوادگی	بهزاد جنتی
شماره دانشجویی	۸۱۰۱۰۳۰۹۸
تاریخ ارسال گزارش	۱۴۰۴/۰۴/۱۰
نام استاد	دکتر هشام فیلی
ترم	بهار ۱۴۰۴

فهرست

سوال ۱ – جستجوی نیمه گمشده: تطبیق تصادفی پیچ و مهره ۵

صورت سوال: ۵

تحلیل مسئله:..... ۵

تحلیل پیچیدگی زمانی ۶

مثال عملی..... ۷

کد الگوریتم RandomizedMatching در پایتون: ۸

سوال ۲ – جستجوی نیمه گمشده: تطبیق تصادفی پیچ و مهره ۹

صورت سوال: ۹

مرحله ۱: تعریف مسئله و شناسایی متغیرهای تصادفی ۹

مرحله ۲: تحلیل توزیع تعداد فرزندان هر خانواده ۹

مرحله ۳: محاسبه امید ریاضی تعداد فرزندان..... ۹

مرحله ۴: تحلیل تعداد پسران و دختران در هر خانواده ۱۰

مرحله ۵: محاسبه نسبت مورد انتظار در کل جامعه..... ۱۰

سوال ۳ – بخت آزمایی در آرایه مرتب..... ۱۱

صورت سوال: ۱۱

الف) پیچیدگی زمانی الگوریتم جستجوی دودویی تصادفی در حالت میانگین..... ۱۲

ب) نوع الگوریتم تصادفی (Las Vegas یا Monte Carlo)..... ۱۴

سوال ۴ – ضرب مشکوک: بازرسی ماتریسی با بازرس تصادفی..... ۱۵

صورت سوال: ۱۵

الف) تحلیل پیچیدگی زمانی الگوریتم تصادفی..... ۱۶

ب) محاسبه کران پایینی احتمال موفقیت الگوریتم در هر تکرار ۱۷

ج) محاسبه احتمال شکست الگوریتم بعد از k بار تکرار و تعیین k مناسب..... ۱۸

سوال ۵- لحظه وصال: وقتی گراف بالاخره همبند می‌شود ۱۹

صورت سوال: ۱۹

سوال ۶- لحظه وصال: وقتی گراف بالاخره همبند می‌شود..... ۲۱

صورت سوال: ۲۱
الگوریتم پیشنهادی ۲۲

شکل‌ها

شکل ۱- کد الگوریتم در پایتون..... ۸

جدول‌ها

No table of figures entries found.

سوال ۱ – جستجوی نیمه گمشده: تطبیق تصادفی پیچ و مهره

صورت سوال:

سوال ۱: جستجوی نیمه‌گمشده: تطبیق تصادفی پیچ و مهره‌ها

یک جعبه پر از پیچ و مهره در اختیار داریم. در این جعبه دقیقا n پیچ و n مهره وجود دارد. می‌دانیم که هر پیچ دقیقا اندازه یک مهره در جعبه است. در هر مقایسه می‌توانیم چک کنیم که آیا یک پیچ به یک مهره می‌خورد یا نه. همچنین در این مقایسه می‌فهمیم که کدام یک از دیگری بزرگ‌تر است. اما حق مقایسه پیچ‌ها با یکدیگر و مهره‌ها با یکدیگر را **نداریم**. الگوریتمی **تصادفی** برای پیدا کردن جفت هر پیچ با زمان اجرای **متوسط** $O(n.lgn)$ طراحی کنید. الگوریتم شما در بدترین حالت چه پیچیدگی زمانی دارد؟

تحلیل مسئله:

داریم: n پیچ و n مهره

هر پیچ دقیقاً با یک مهره تطبیق دارد

می‌توانیم پیچ را با مهره مقایسه کنیم (تطبیق یا عدم تطبیق + بزرگتر/کوچکتر)

نمی‌توانیم پیچ‌ها را با هم یا مهره‌ها را با هم مقایسه کنیم

درک مسئله و محدودیت‌ها

این مسئله یک نوع خاص از مسائل مرتب‌سازی است که با محدودیت‌های خاصی روبرو هستیم. ما n پیچ و n مهره داریم که هر پیچ دقیقاً با یک مهره تطبیق دارد. تنها عملیات مجاز، مقایسه یک پیچ با یک مهره است که نتیجه آن سه حالت دارد: پیچ کوچک‌تر، برابر، یا بزرگ‌تر از مهره. هدف ما پیدا کردن تمام جفت‌های تطبیقی با زمان متوسط O(n log n) است. این محدودیت باعث می‌شود که نتوانیم از روش‌های معمول مرتب‌سازی استفاده کنیم.

ایده اصلی الگوریتم

الگوریتم ما بر اساس استراتژی تقسیم و حل (Divide and Conquer) کار می‌کند، اما با استفاده از تصادفی‌سازی برای انتخاب عنصر محوری. ایده کلی این است که یک پیچ را به صورت تصادفی انتخاب کرده، با استفاده از آن مهره‌ها را تقسیم‌بندی کنیم، سپس مهره جفت آن پیچ را پیدا کرده و با استفاده از آن مهره، پیچ‌های باقی‌مانده را نیز تقسیم‌بندی کنیم. این فرآیند به صورت بازگشتی ادامه می‌یابد تا تمام جفت‌ها پیدا شوند.

مرحله ۱: انتخاب تصادفی پیچ محوری

در ابتدای هر فراخوانی بازگشتی، یک پیچ را به صورت کاملاً تصادفی از مجموعه پیچ‌های باقی‌مانده انتخاب می‌کنیم و آن را به عنوان pivot قرار می‌دهیم. این انتخاب تصادفی کلید اصلی عملکرد خوب الگوریتم است، زیرا احتمال انتخاب یک عنصر بد (که منجر به تقسیم‌بندی نامتعادل شود) را کاهش می‌دهد. در حالت ایده‌آل، این پیچ محوری باید مجموعه را به دو قسمت تقریباً مساوی تقسیم کند تا عمق بازگشت کم باشد.

مرحله ۲: تقسیم‌بندی مهره‌ها بر اساس پیچ محوری

پس از انتخاب پیچ محوری، تمام مهره‌های موجود را با این پیچ مقایسه می‌کنیم. این مقایسه‌ها مهره‌ها را به سه دسته تقسیم می‌کند: مهره‌هایی که کوچک‌تر از پیچ محوری هستند، مهره‌ای که دقیقاً برابر پیچ محوری است (جفت آن)، و مهره‌هایی که بزرگ‌تر از پیچ محوری هستند. از آنجا که هر پیچ دقیقاً یک جفت دارد، می‌دانیم که حتماً یکی از مهره‌ها با پیچ محوری برابر خواهد بود. این مرحله O(n) مقایسه نیاز دارد.

مرحله ۳: شناسایی مهره جفت و تشکیل اولین جفت

از میان مهره‌هایی که با پیچ محوری برابر هستند (که فقط یکی وجود دارد)، آن مهره را به عنوان جفت پیچ محوری شناسایی می‌کنیم. این اولین جفت کاملی است که پیدا کرده‌ایم و آن را در لیست نتایج قرار می‌دهیم. حالا مسئله اصلی به دو زیرمسئله کوچک‌تر تبدیل شده است: تطبیق پیچ‌ها و مهره‌های کوچک‌تر، و تطبیق پیچ‌ها و مهره‌های بزرگ‌تر.

مرحله ٤: تقسیم‌بندی پیچ‌های باقی‌مانده

حالا باید پیچ‌های باقی‌مانده (به جز پیچ محوری که جفتش پیدا شده) را نیز تقسیم‌بندی کنیم. برای این کار از مهره جفت پیچ محوری استفاده می‌کنیم. تمام پیچ‌های باقی‌مانده را با این مهره مقایسه کرده و آنها را به دو دسته تقسیم می‌کنیم: پیچ‌هایی که کوچک‌تر از مهره محوری هستند و پیچ‌هایی که بزرگ‌تر هستند. این تقسیم‌بندی نیز O(n-1) مقایسه نیاز دارد.

مرحله ٥: حل بازگشتی زیرمسائل

پس از تقسیم‌بندی، دو زیرمسئله مستقل داریم: یکی شامل پیچ‌ها و مهره‌های کوچک‌تر، و دیگری شامل پیچ‌ها و مهره‌های بزرگ‌تر. هر کدام از این زیرمسائل را به صورت بازگشتی با همین الگوریتم حل می‌کنیم. این فرآیند تا زمانی ادامه می‌یابد که به مسائل پایه (n=1) برسیم که در آن فقط یک پیچ و یک مهره داریم و آنها را مستقیماً جفت می‌کنیم.

تحلیل پیچیدگی زمانی

حالت متوسط: O(n log n)

در حالت متوسط، انتظار داریم که پیچ محوری تصادفی، مجموعه را به دو قسمت تقریباً مساوی تقسیم کند. اگر $T(n)$ زمان حل مسئله با اندازه n باشد، داریم:

زمان تقسیم‌بندی: $O(n)$

دو زیرمسئله با اندازه تقریبی $n/2$

بنابراین: $T(n) = 2T(n/2) + O(n)$ که طبق قضیه Master، $T(n) = O(n \log n)$

بدترین حالت: $O(n^2)$

اگر در هر مرحله بدترین پیچ (کوچک‌ترین یا بزرگ‌ترین) انتخاب شود، یکی از زیرمسائل خالی و دیگری شامل n-1 عنصر خواهد بود. در این صورت: $T(n) = T(n-1) + O(n) = O(n^2)$

مثال عملی

فرض کنید ٤ پیچ {P1, P2, P3, P4} و ٤ مهره {N1, N2, N3, N4} داریم که:

- P1 ↔ N2 (کوچک‌ترین)
- P2 ↔ N4 (دومین کوچک)
- P3 ↔ N1 (دومین بزرگ)
- P4 ↔ N3 (بزرگ‌ترین)

مرحله ۱: پیچ P3 را تصادفی انتخاب می‌کنیم.

مرحله ۲: مهره‌ها را با P3 مقایسه می‌کنیم:

- N1 = P3 (جفت)
- P3 > N2 (کوچک‌تر)
- P3 < N3 (بزرگ‌تر)
- P3 > N4 (کوچک‌تر)

مرحله ۳: جفت (P3, N1) را پیدا کردیم.

مرحله ٤: پیچ‌های باقی‌مانده را با N1 مقایسه می‌کنیم:

- N1 > P1 (کوچک‌تر)
- N1 > P2 (کوچک‌تر)
- N1 < P4 (بزرگ‌تر)

مرحله ۵: دو زیرمسئله داریم:

- کوچک‌تر: {P1, P2} و {N2, N4}
- بزرگ‌تر: {P4} و {N3}

زیرمسئله بزرگ‌تر فوراً حل می‌شود: (P4, N3)

برای زیرمسئله کوچک‌تر، P1 را انتخاب می‌کنیم، N2 را جفت آن پیدا می‌کنیم، و در نهایت (P2, N4) باقی می‌ماند.

نتیجه نهایی: {(P1,N2), (P2,N4), (P3,N1), (P4,N3)}

این الگوریتم با ترکیب تصادفی‌سازی و تقسیم و حل، عملکرد متوسط بهتری نسبت به روش‌های قطعی دارد و برای مسائل بزرگ بسیار کارآمد است.

کد الگوریتم RandomizedMatching در پایتون:

۱. اگر n = 1، پیچ و مهره را با هم تطبیق بده

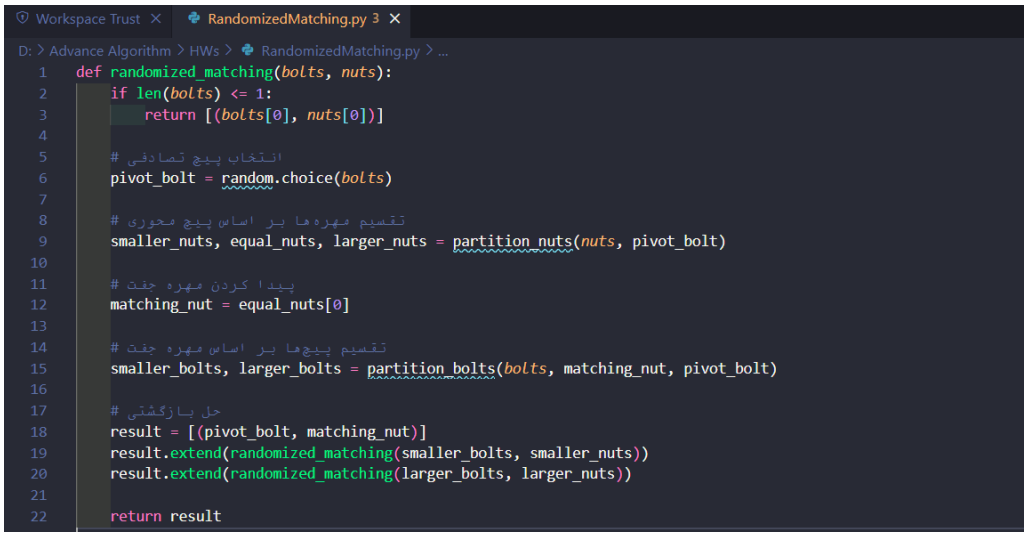
۲. یک پیچ تصادفی p را انتخاب کن

۳. با استفاده از p، مهره‌ها را به سه دسته تقسیم کن:

- کوچک‌تر از p
- برابر با p (جفت p)
- بزرگ‌تر از p

۴. با استفاده از مهره جفت p، پیچ‌ها را نیز تقسیم کن

۵. بازگشتی روی زیرمسائل کوچک‌تر و بزرگ‌تر اعمال کن




```python
def randomized_matching(bolts, nuts):
    if len(bolts) <= 1:
        return [(bolts[0], nuts[0])]

    # انتخاب پیچ تصادفی
    pivot_bolt = random.choice(bolts)

    # تقسیم مهره ها بر اساس پیچ محوری
    smaller_nuts, equal_nuts, larger_nuts = partition_nuts(nuts, pivot_bolt)

    # پیدا کردن مهره جفت
    matching_nut = equal_nuts[0]

    # تقسیم پیچ‌ها بر اساس مهره جفت
    smaller_bolts, larger_bolts = partition_bolts(bolts, matching_nut, pivot_bolt)

    # حل بازگشتی
    result = [(pivot_bolt, matching_nut)]
    result.extend(randomized_matching(smaller_bolts, smaller_nuts))
    result.extend(randomized_matching(larger_bolts, larger_nuts))

    return result
```

شکل ۱- کد الگوریتم در پایتون

سوال ۲ – جستجوی نیمه گمشده: تطبیق تصادفی پیچ و مهره

صورت سوال:

سوال ۲: ملت پسرپرست: امید ریاضی تولدها در سرزمین نابرابر

فرهنگ یک کشور عجیب به این صورت است که خانواده‌ها می‌خواهند حتما فرزند پسر داشته باشند. در نتیجه فرآیند فرزندآوری را وقتی که یک پسر متولد شود، ادامه می‌دهند و بعد از آن متوقف می‌کنند!

با فرض اینکه احتمال پسر و دختر بودن فرزند جدید برابر است، مقدار مورد انتظار نسبت تعداد پسرها به تعداد دخترها در این کشور را بدست آورید.

مرحله ۱: تعریف مسئله و شناسایی متغیرهای تصادفی

در این مسئله با کشوری مواجه هستیم که خانواده‌ها تا زمانی که یک پسر به دنیا نیاورند، به فرزندآوری ادامه می‌دهند و پس از تولد اولین پسر، دیگر فرزند نمی‌آورند. با فرض اینکه احتمال تولد پسر و دختر برابر و مستقل از یکدیگر است (هر کدام ۰/۵)، باید نسبت مورد انتظار تعداد پسران به دختران را در کل جامعه محاسبه کنیم.

برای هر خانواده، متغیر تصادفی X را تعداد کل فرزندان آن خانواده تعریف می‌کنیم که شامل یک پسر (آخرین فرزند) و تعدادی دختر (فرزندان قبلی) است.

مرحله ۲: تحلیل توزیع تعداد فرزندان هر خانواده

تعداد فرزندان هر خانواده از توزیع هندسی پیروی می‌کند، زیرا خانواده‌ها تا زمان وقوع اولین "موفقیت" (تولد پسر) به فرزندآوری ادامه می‌دهند. در توزیع هندسی با پارامتر p، متغیر تصادفی نشان‌دهنده تعداد آزمایش‌های مورد نیاز تا اولین موفقیت است.

در اینجا $p = 0.5$ (احتمال تولد پسر) است. تابع جرم احتمال توزیع هندسی به صورت زیر است:

$$P(X = k) = (1 - p)^{k-1} \times p$$

برای $k \geq 1$. در این مسئله داریم:

$$P(X = k) = (0.5)^{k-1} \times 0.5 = (0.5)^k$$

برای $k \geq 1$، که احتمال اینکه یک خانواده دقیقاً k فرزند داشته باشد را نشان می‌دهد.

مرحله ۳: محاسبه امید ریاضی تعداد فرزندان

امید ریاضی متغیر تصادفی X در توزیع هندسی با پارامتر p برابر است با:

$$E[X] = \frac{1}{p}$$

در اینجا با p = 0.5، داریم:

$$E[X] = \frac{1}{0.5} = 2$$

این بدان معناست که به طور متوسط، هر خانواده ۲ فرزند خواهد داشت. می‌توانیم این را به صورت جبری نیز اثبات کنیم:

$$E[X] = \sum_{k=1}^{\infty} k \times P(X = k) = \sum_{k=1}^{\infty} k \times (0.5)^k = 0.5 \times \sum_{k=1}^{\infty} k \times (0.5)^{k-1}$$
$$= 0.5 \times \frac{1}{(1-0.5)^2} = 0.5 \times 4 = 2$$

مرحله ٤: تحلیل تعداد پسران و دختران در هر خانواده

در هر خانواده، تعداد پسران همیشه ثابت و برابر ۱ است (آخرین فرزند). تعداد دختران در هر خانواده برابر با (X-1) است، زیرا از کل X فرزند، یکی پسر است و بقیه دختر هستند. بنابراین:

تعداد پسران در هر خانواده = ۱

تعداد دختران در هر خانواده = X - 1

امید ریاضی تعداد دختران در هر خانواده:

$$E[\text{تعداد دختران}] = E[X-1] = E[X] - 1 = 2 - 1 = 1$$

مرحله ٥: محاسبه نسبت مورد انتظار در کل جامعه

اگر تعداد کل خانواده‌ها در این کشور را n در نظر بگیریم:

تعداد کل پسران = n × 1 = n (هر خانواده دقیقاً یک پسر دارد)

تعداد کل دختران = n × E[تعداد دختران در هر خانواده] = n × 1 = n

بنابراین نسبت مورد انتظار تعداد پسران به دختران در کل جامعه:

$$E\left[\text{نسبت پسران به دختران}\right] = \frac{n}{n} = 1$$

مرحله ٦: تفسیر نتیجه

این نتیجه جالب نشان می‌دهد که علی‌رغم فرهنگ پسرپرستی و تمایل خانواده‌ها به داشتن فرزند پسر، در سطح کلان جامعه، نسبت پسران به دختران به طور متوسط برابر با ۱ خواهد بود.

این یک نتیجه شهودی نیست، زیرا ممکن است انتظار داشته باشیم در چنین جامعه‌ای تعداد پسران بیشتر از دختران باشد، اما محاسبات ریاضی نشان می‌دهد که توزیع جنسیتی در کل جامعه متعادل باقی می‌ماند.

این نتیجه از این واقعیت ناشی می‌شود که هر خانواده به طور متوسط یک دختر قبل از تولد پسر خواهد داشت، که دقیقاً با تعداد پسران (یک پسر برای هر خانواده) برابر است. بنابراین، در مقیاس بزرگ جامعه، تعداد کل پسران و دختران تقریباً برابر خواهد بود، و نسبت آنها ۱ می‌شود.

سوال ۳ – بخت آزمایی در آرایه مرتب

صورت سوال:

سوال ۳: بخت‌آزمایی در آرایه مرتب

می‌دانیم الگوریتم Binary Search برای پیدا کردن سریع یک عدد در آرایه مرتب‌شده استفاده می‌شود. حال در نسخه تصادفی آن به جای اینکه هر بار عدد را با وسط بازه فعلی مقایسه بکنیم، با یک اندیس تصادفی که با احتمال برابر از بازه فعلی انتخاب می‌شود مقایسه می‌کنیم. در نتیجه هر بار بازه جستجو نصف نمی‌شود. جستجو به صورت بازگشتی ادامه پیدا می‌کند تا به یک عنصر برسیم.

۳

الف) پیچیدگی زمانی الگوریتم جستجوی دودویی تصادفی را در حالت میانگین تحلیل کنید. لازم است جزییات ریاضی محاسبه خود را ذکر کنید.

ب) تحقیق کنید این الگوریتم تصادفی از دسته Las Vegas است یا Monte Carlo؟ هر کدام را توضیح دهید.

الف) پیچیدگی زمانی الگوریتم جستجوی دودویی تصادفی در حالت میانگین

برای محاسبه دقیق پیچیدگی زمانی این الگوریتم، به صورت گام به گام پیش می‌رویم:

تعریف مسئله

در جستجوی دودویی تصادفی، به جای انتخاب عنصر وسط در هر مرحله، یک عنصر به صورت تصادفی و با توزیع یکنواخت از بازه جستجوی فعلی انتخاب می‌شود. فرض می‌کنیم آرایه مرتب $A[۱...n]$ را داریم و می‌خواهیم عنصر x را پیدا کنیم.

تعریف متغیر تصادفی

متغیر تصادفی $T(n)$ را تعریف می‌کنیم که نشان‌دهنده تعداد مقایسه‌های مورد نیاز برای جستجو در آرایه با n عنصر است. هدف ما محاسبه $E[T(n)]$(امید ریاضی $T(n)$) است.

حالت پایه

برای n=۱، تنها یک عنصر داریم و فقط یک مقایسه انجام می‌شود. پس: $T(۱) = ۱$

رابطه بازگشتی

برای $n > ۱$، فرض کنیم عنصر انتخاب شده در موقعیت i باشد (که با احتمال $۱/n$ برای هر $i \in \{۱,۲,...,n\}$ انتخاب می‌شود). سه حالت ممکن است:

$A[i] = x$: جستجو تمام می‌شود (۱ مقایسه)

$A[i] > x$: جستجو در زیربازه $A[۱...i-۱]$ ادامه می‌یابد ($۱ + T(i-۱)$ مقایسه)

$A[i] < x$: جستجو در زیربازه $A[i+۱...n]$ ادامه می‌یابد ($۱ + T(n-i)$ مقایسه)

با فرض اینکه عنصر مورد جستجو در آرایه وجود ندارد (بدترین حالت)، رابطه بازگشتی به این صورت خواهد بود:

$$E[T(n)] = 1 + 1/n \sum_{i=1}^{n} max\,(E[T(i-1)], E[T(n-i)])$$

اما به دلیل پیچیدگی محاسبه max ، می‌توانیم حالتی را در نظر بگیریم که در آن عنصر مورد جستجو به صورت تصادفی در آرایه قرار دارد یا خارج از آرایه است. در این صورت:

$$E[T(n)] = 1 + 1/n \sum_{i=1}^{n} (E[T(i-1)] + E[T(n-i)]) \cdot 1/2$$

که می‌توان آن را به صورت زیر ساده کرد:

$$E[T(n)] = 1 + 1/2n \sum_{i=1}^{n} (E[T(i-1)] + E[T(n-i)])$$

ساده‌سازی رابطه بازگشتی

با تغییر متغیر $j = i - ۱$ در جمع اول و $j = n - i$ در جمع دوم:

$$E[T(n)] = 1 + 1/2n \sum_{j=0}^{n-1} E[T(j)] + 1/2n \sum_{j=0}^{n-1} E[T(j)]$$

$$E[T(n)] = 1 + 1/n \sum_{j=0}^{n-1} E[T(j)]$$

حل رابطه بازگشتی

برای حل این رابطه، از روش تفاضل استفاده می‌کنیم. ابتدا رابطه را برای $n - ۱$ می‌نویسیم:

$$E[T(n-1)] = 1 + 1/n - 1 \sum_{j=0}^{n-2} E[T(j)]$$

سپس $n \cdot E[T(n)]$ را محاسبه می‌کنیم:

$$n \cdot E[T(n)] = n + \sum_{j=0}^{n-1} E[T(j)]$$

و $(n - ۱) \cdot E[T(n - ۱)]$ را نیز محاسبه می‌کنیم:

$$(n-1) \cdot E[T(n-1)] = (n-1) + \sum_{j=0}^{n-2} E[T(j)]$$

با تفاضل این دو رابطه:

$$n \cdot E[T(n)] - (n-1) \cdot E[T(n-1)] = n - (n-1) + E[T(n-1)]$$

$$n \cdot E[T(n)] - (n-1) \cdot E[T(n-1)] = 1 + E[T(n-1)]$$

$$n \cdot E[T(n)] = (n-1) \cdot E[T(n-1)] + 1 + E[T(n-1)]$$

$$n \cdot E[T(n)] = n \cdot E[T(n-1)] + 1$$

$$E[T(n)] = E[T(n-1)] + \frac{1}{n}$$

این یک رابطه بازگشتی ساده‌تر است که با توجه به شرط پایه $E[T(۱)] = ۱$، می‌توانیم آن را به صورت زیر حل کنیم:

$$E[T(n)] = E[T(1)] + \sum_{i=2}^{n} \frac{1}{i} = 1 + \sum_{i=2}^{n} \frac{1}{i} = \sum_{i=1}^{n} \frac{1}{i} = Hn$$

که Hn عدد هارمونیک n-ام است.

نتیجه نهایی

می‌دانیم که $\gamma \approx$ ۰.۵۷۷۲۱ که $Hn \approx ln\,(n) + \gamma$ ثابت اویلر-ماسکرونی است. بنابراین:

$$E[T(n)] = Hn \approx ln\,(n) + O(1)$$

بنابراین، پیچیدگی زمانی الگوریتم جستجوی دودویی تصادفی در حالت میانگین $O(log\ n)$ است.

ب) نوع الگوریتم تصادفی (Las Vegas یا Monte Carlo)

این الگوریتم از نوع **Las Vegas** است. دلایل:

۱. الگوریتم‌های Las Vegas همیشه پاسخ صحیح را تولید می‌کنند، اما زمان اجرای آنها متغیر و تصادفی است. در این الگوریتم، ما همیشه عنصر مورد نظر را پیدا می‌کنیم (اگر در آرایه موجود باشد) یا مطمئن می‌شویم که در آرایه وجود ندارد.

۲. در مقابل، الگوریتم‌های Monte Carlo ممکن است گاهی پاسخ اشتباه تولید کنند، اما زمان اجرای آنها معمولاً ثابت است. در الگوریتم جستجوی دودویی تصادفی، احتمال خطا صفر است و پاسخ همیشه درست است.

۳. در این الگوریتم، انتخاب تصادفی فقط روی زمان اجرا تأثیر می‌گذارد، نه روی صحت نتیجه. ممکن است در برخی اجراها، الگوریتم سریع‌تر به نتیجه برسد (مثلاً اگر در اولین انتخاب تصادفی، دقیقاً عنصر مورد نظر را پیدا کند) و در برخی اجراها کندتر باشد (مثلاً اگر همیشه عناصر نزدیک به انتهای بازه را انتخاب کند).

بنابراین، الگوریتم جستجوی دودویی تصادفی یک الگوریتم Las Vegas است که همیشه پاسخ درست می‌دهد، اما زمان اجرای آن متغیر است و به صورت میانگین $O(log\ n)$خواهد بود.

این الگوریتم از نوع **Las Vegas** است. به دلایل زیر:

۱. **صحت نتیجه**: الگوریتم‌های Las Vegas همیشه پاسخ صحیح تولید می‌کنند، اما زمان اجرایشان متغیر است. در جستجوی دودویی تصادفی، الگوریتم همیشه پاسخ صحیح را بر می‌گرداند، یعنی:

- اگر عنصر در آرایه باشد، آن را پیدا می‌کند.
- اگر عنصر در آرایه نباشد، عدم وجود آن را گزارش می‌کند.

۲. **زمان اجرای متغیر**: زمان اجرای الگوریتم متغیر است و به انتخاب‌های تصادفی بستگی دارد. ممکن است در بهترین حالت، اولین انتخاب تصادفی دقیقاً عنصر مورد نظر باشد (پیچیدگی $O(۱)$)، یا در بدترین حالت، هر انتخاب تصادفی ما را به زیربازه‌های کوچک‌تری هدایت کند (پیچیدگی $O(n)$).

۳. **مقایسه با Monte Carlo**: در الگوریتم‌های Monte Carlo، زمان اجرا معمولاً قطعی است، اما نتیجه ممکن است با احتمال کوچکی نادرست باشد. در جستجوی دودویی تصادفی، نتیجه همیشه درست است، اما زمان اجرا متغیر است.

٤. **استفاده از تصادفی‌سازی**: تصادفی‌سازی در این الگوریتم فقط برای انتخاب عنصر مورد بررسی در هر مرحله استفاده می‌شود، نه برای تصمیم‌گیری درباره خروجی الگوریتم.

به طور خلاصه، جستجوی دودویی تصادفی یک الگوریتم Las Vegas کلاسیک است که همیشه نتیجه صحیح تولید می‌کند، اما زمان اجرای آن متغیر و تصادفی است. در حالت میانگین، پیچیدگی زمانی آن $O(log\ n)$ است، اما این پیچیدگی از جستجوی دودویی استاندارد (با پیچیدگی زمانی بدترین حالت $\lceil log\ ۲n \rceil$) بیشتر است.

سوال ۴ – ضرب مشکوک: بازرسی ماتریسی با بازرس تصادفی

صورت سوال:

سوال ۴: ضرب مشکوک: بازرسی ماتریسی با بازرس تصادفی

فرض کنید سه ماتریس $n \times n$ A، B و C به عنوان ورودی داده شده است و می‌خواهیم الگوریتمی طراحی کنیم که چک کند آیا حاصل ضرب ماتریسی $A * B$ برابر C می‌شود یا نه. یک راه‌حل بدیهی آن است که با پیچیدگی زمانی $O(n^3)$ حاصل ضرب دو ماتریس را محاسبه کرده و با C مقایسه کنیم. اما الگوریتم دیگری با رویکرد تصادفی راه‌حل سریع‌تری ارائه می‌دهد. به این صورت که ابتدا بردار یک بعدی r به طول n به صورت تصادفی با مقادیر ۰ و ۱ می‌سازد. سپس حاصل عبارت زیر را محاسبه می‌کند:

$$A * (B * r) - (C * r)$$

اگر همه عناصر بردار حاصل صفر نشود، یعنی حاصل ضرب ماتریسی برابر C نیست. ولی اگر همه عناصر آن نیز برابر ۰ شود، ممکن است همچنان حاصل ضرب برابر C نباشد. این الگوریتم را k بار تکرار کرده و فقط اگر در هر k بار همه عناصر برابر ۰ شوند، خروجی مثبت می‌دهیم.

الف) تحلیل پیچیدگی زمانی الگوریتم تصادفی

الگوریتم ارائه شده یک روش هوشمندانه برای بررسی تساوی $A \times B = C$ بدون نیاز به محاسبه کامل حاصل‌ضرب ماتریس‌ها است. در روش معمولی، محاسبه حاصل‌ضرب دو ماتریس n×n نیازمند $O(n^3)$ عملیات است، اما الگوریتم تصادفی ارائه شده با استفاده از یک بازرس تصادفی، این پیچیدگی را به طور قابل توجهی کاهش می‌دهد.

مراحل الگوریتم و تحلیل پیچیدگی زمانی آن به شرح زیر است:

تولید بردار تصادفی r: در این مرحله، یک بردار تصادفی r با طول n تولید می‌کنیم که هر عنصر آن با احتمال برابر ۰ یا ۱ است. این مرحله نیازمند $O(n)$ عملیات است، زیرا باید برای هر یک از n عنصر بردار، یک مقدار تصادفی تولید کنیم.

محاسبه $B \times r$: در این مرحله، حاصل‌ضرب ماتریس B در بردار r را محاسبه می‌کنیم. برای این کار، باید هر سطر ماتریس B را در بردار r ضرب کنیم. هر سطر ماتریس B دارای n عنصر است و برای ضرب یک سطر در بردار r، نیاز به $O(n)$ عملیات داریم. با توجه به اینکه ماتریس B دارای n سطر است، پیچیدگی این مرحله $O(n^2)$ خواهد بود.

محاسبه $A \times (B \times r)$: در این مرحله، حاصل‌ضرب ماتریس A در بردار $B \times r$ را محاسبه می‌کنیم. مشابه مرحله قبل، این مرحله نیز دارای پیچیدگی $O(n^2)$ است.

محاسبه r×C: همزمان با مراحل قبل یا بعد از آن، حاصل‌ضرب ماتریس C در بردار r را محاسبه می‌کنیم که این مرحله نیز دارای پیچیدگی $O(n^2)$ است.

محاسبه $A \times (B \times r) - (C \times r)$: در این مرحله، تفاضل دو بردار حاصل از مراحل ۳ و ٤ را محاسبه می‌کنیم. این مرحله نیازمند $O(n)$ عملیات است، زیرا باید تفاضل n جفت عنصر را محاسبه کنیم.

بررسی صفر بودن همه عناصر بردار حاصل: در این مرحله، بررسی می‌کنیم که آیا تمام عناصر بردار حاصل از مرحله ٥ برابر صفر هستند یا خیر. این مرحله نیز نیازمند $O(n)$ عملیات است.

تکرار الگوریتم k بار: با توجه به اینکه الگوریتم را k بار تکرار می‌کنیم، پیچیدگی زمانی کل برابر با k ضربدر مجموع پیچیدگی‌های مراحل ۱ تا ٦ خواهد بود.

بنابراین، پیچیدگی زمانی کل الگوریتم برابر است با: $O(k \times (n + n^2 + n^2 + n^2 + n + n)) =$
$O(k \times (۳n^2 + ۳n)) = O(k \times n^2)$

از آنجایی که معمولاً k یک مقدار ثابت و نسبتاً کوچک است (مثلاً در بخش (ج) خواهیم دید که $k = ۷$ کافی است)، می‌توانیم پیچیدگی نهایی را به صورت $O(n^2)$ بیان کنیم که بسیار بهتر از الگوریتم قطعی با پیچیدگی $O(n^3)$ است.

این بهبود پیچیدگی به دلیل استفاده هوشمندانه از خاصیت خطی بودن عملیات ماتریسی است. در واقع، به جای محاسبه کامل $A \times B$ و مقایسه با C، ما از یک بردار تصادفی برای نمونه‌گیری از فضای ماتریس‌ها استفاده می‌کنیم و با احتمال بالایی می‌توانیم تشخیص دهیم که آیا $A \times B = C$ است یا خیر.

ب) محاسبه کران پایینی احتمال موفقیت الگوریتم در هر تکرار

برای تحلیل احتمال موفقیت الگوریتم، ابتدا به بررسی دقیق عملکرد آن می‌پردازیم. الگوریتم ما عبارت $A \times (B \times r) - (C \times r)$ را محاسبه می‌کند و اگر این عبارت برابر بردار صفر باشد، نتیجه می‌گیریم که احتمالاً $A \times B = C$ است.

با استفاده از خاصیت توزیع‌پذیری ضرب ماتریسی، می‌توانیم این عبارت را به صورت زیر بازنویسی کنیم: $A \times (B \times r) - (C \times r) = (A \times B) \times r - C \times r = (A \times B - C) \times r$

حال اگر $D = A \times B - C$ تعریف کنیم، عبارت بالا به صورت r×D خواهد بود.

اکنون دو حالت را در نظر می‌گیریم:

اگر $A \times B = C$ باشد، آنگاه $D = ۰$ (ماتریس صفر) و در نتیجه $D \times r = ۰$ خواهد بود. در این حالت، الگوریتم همیشه جواب درست می‌دهد (یعنی احتمال موفقیت ۱ است).

اگر $A \times B \neq C$ باشد، آنگاه $D \neq ۰$ (یعنی حداقل یک درایه غیر صفر در D وجود دارد). در این حالت، باید احتمال اینکه $D \times r = ۰$ شود را محاسبه کنیم.

برای محاسبه احتمال در حالت دوم، از قضیه زیر در جبر خطی استفاده می‌کنیم:

قضیه :اگر Dیک ماتریس غیر صفر $n \times n$ باشد و rیک بردار تصادفی با عناصر مستقل ۰ و ۱ (با احتمال برابر) باشد، آنگاه :

$$P(D \times r = 0) \leq 1/2$$

اثبات :فرض کنیم Dیک ماتریس غیر صفر است. پس حداقل یک سطر غیر صفر در D وجود دارد. فرض کنیم سطر i از Dغیر صفر است و آن را با Di نشان می‌دهیم.

حاصل‌ضرب سطر i از D در بردار r برابر است با:

$$Di \times r = \sum_{j=1}^{n} Dij \times rj$$

با توجه به اینکه سطر Di غیر صفر است، حداقل یک j وجود دارد که $Dij \neq ۰$ است. فرض کنیم $j۱, j۲, \ldots, jk$ اندیس‌های ستون‌هایی هستند که در آنها $Dij \neq ۰$ است.

برای اینکه $Di \times r = ۰$ شود، باید داشته باشیم: $\sum_{l=1}^{k} D_{ijl} \times r_{jl} = 0$

با توجه به اینکه rjl ها مستقل و با احتمال برابر ۰ یا ۱ هستند، و حداقل یکی از $Dijl$ها غیر صفر است، می‌توانیم از قضیه پایه در جبر خطی استفاده کنیم که می‌گوید:

اگر $v۱, v۲, \ldots, vk$ بردارهای غیر صفر در یک فضای برداری باشند، آنگاه برای هر بردار تصادفی r با عناصر مستقل و با توزیع یکنواخت، احتمال اینکه $\sum_{i=1}^{k} vi \times ri = 0$ شود، حداکثر ۱/۲ است.

در نتیجه، احتمال اینکه $Di \times r = ۰$ شود، حداکثر ۱۲ است. و از آنجایی که برای اینکه $D \times r =$ ۰ شود، باید برای تمام سطرهای D داشته باشیم $Di \times r = ۰$، پس: $P(D \times r = ۰) \leq ۱/۲$

بنابراین، احتمال اینکه الگوریتم در یک تکرار، تشخیص درست دهد که $A \times B \neq C$ است (یعنی احتمال موفقیت)، حداقل ۱/۲ = ۱/۲ − ۱ است.

ج) محاسبه احتمال شکست الگوریتم بعد از k بار تکرار و تعیین k مناسب

با توجه به نتایج بخش (ب)، می‌دانیم که در هر تکرار الگوریتم، احتمال شکست (یعنی احتمال اینکه الگوریتم تشخیص ندهد که $A \times B \neq C$ است) حداکثر ۱/۲ است.

از آنجایی که تکرارهای الگوریتم مستقل از یکدیگر هستند (زیرا در هر تکرار، یک بردار تصادفی جدید و مستقل r تولید می‌شود)، احتمال اینکه الگوریتم در تمام k تکرار شکست بخورد برابر است با:

$$P(\text{شکست در تمام } k \text{ تکرار}) = (1/2)^k$$

برای اینکه احتمال شکست کلی الگوریتم کمتر از ۱٪ باشد، باید داشته باشیم:

$$(1/2)^k < 0.01$$

با گرفتن لگاریتم در مبنای ۲ از طرفین نامساوی:

$$k \times \log_2(\frac{1}{2}) < \log_2(0.01)$$

با توجه به اینکه $\log_2(1/2) = -1$ و $\log_2(0.01) = \log_2(10^{-2}) = -2 \times \log_2(10) \approx$ $-2 \times 3.32 = -6.64$ ، خواهیم داشت:

$$-k < -6.64 \; k > 6.64$$

بنابراین، برای اینکه احتمال شکست کلی الگوریتم کمتر از ۱٪ باشد، باید k حداقل ۷ باشد. یعنی الگوریتم را باید حداقل ۷ بار تکرار کنیم.

محاسبه دقیق‌تر: $\log_2(0.01) = \log_2(10^{-2}) = -2 \times \log_2(10) = -2 \times log\ (10)/$ $log\ (2) = -2 \times 2.303/0.693 \approx -6.64$

با $k =$ ۷ ، احتمال شکست برابر است با: $(1/2)^7 = 1/128 \approx 0.0078 < 0.01$

که کمتر از ۱٪ است، پس $k = ۷$ کافی است.

این نتیجه نشان می‌دهد که با تنها ۷ بار تکرار الگوریتم تصادفی، می‌توانیم با اطمینان بیش از ۹۹٪ تشخیص دهیم که آیا $A \times B = C$ است یا خیر. این در حالی است که پیچیدگی زمانی الگوریتم ما $O(۷ \times n^2) = O(n^2)$ است، که بسیار بهتر از الگوریتم قطعی با پیچیدگی $O(n^3)$ است.

این مثال به خوبی نشان می‌دهد که چگونه الگوریتم‌های تصادفی می‌توانند با قربانی کردن قطعیت (با احتمال بسیار کم)، پیچیدگی زمانی را به طور قابل توجهی کاهش دهند.

سوال ۵- لحظه وصال: وقتی گراف بالاخره همبند می‌شود

صورت سوال:

سوال ۵: لحظه‌ی وصال: وقتی گراف بالاخره همبند می‌شود

فرض کنید یک گراف ساده با n راس برچسب‌گذاری شده داریم. در ابتدا هیچ یالی در این گراف وجود ندارد. سپس در $\frac{n.(n-1)}{2}$ مرحله یال‌های گراف را اضافه می‌کنیم. یعنی در هر مرحله یکی از یال‌ها بین دو راسی که تا الان متصل نشده‌اند، به صورت تصادفی انتخاب و به گراف اضافه می‌شود.

امید ریاضی تعداد مراحلی که طول می‌کشد تا گراف همبند شود را محاسبه کنید. لازم است متغیر تصادفی، قضایا و جزییات ریاضی راه‌حل خود را توضیح دهید. چون محاسبه دقیق تعداد یال‌های بین مولفه‌ها در یک مرحله دشوار است، از **تخمین** آن در محاسبات خود استفاده کنید.

مسئله: محاسبه امید ریاضی تعداد یال‌هایی که باید به گراف با n راس اضافه شود تا برای اولین بار همبند شود.

مرحله اول: تعریف متغیر تصادفی و تجزیه مسئله

تعریف متغیر اصلی:

X = تعداد کل یال‌های مورد نیاز برای همبند شدن گراف

استراتژی حل: فرآیند همبند شدن را به مراحل کوچک‌تر تقسیم می‌کنیم. در ابتدا n مولفه همبندی داریم (هر راس یک مولفه) و باید به ۱ مولفه برسیم.

تعریف متغیرهای کمکی:

X_i= تعداد یال‌های اضافی مورد نیاز برای کاهش تعداد مولفه‌های همبندی از i به i-1

بنابراین: $X = X_n + X_(n-1) + \ldots + X_2$

مرحله دوم: استفاده از خطی بودن امید ریاضی

از آنجا که امید ریاضی خطی است: $E[X] = E[X_n] + E[X_(n-1)] + \ldots + E[X_2] = \Sigma(i = 2\ to\ n)\ E[X_i]$

حال باید $E[X_i]$ را برای هر i محاسبه کنیم.

مرحله سوم: تحلیل X_i و توزیع هندسی

ماهیت X_i: وقتی i مولفه همبندی داریم، X_i نشان‌دهنده تعداد یال‌هایی است که باید اضافه کنیم تا اولین یال "مفید" را پیدا کنیم (یالی که دو مولفه مختلف را به هم وصل کند).

توزیع X_i: X_i دارای توزیع هندسی است، زیرا:

- هر یال اضافه شده یک آزمایش مستقل است
- احتمال موفقیت (پیدا کردن یال مفید) در هر آزمایش p_i است
- ما به دنبال اولین موفقیت هستیم

برای توزیع هندسی با احتمال موفقیت p_i: $E[X_i] = 1/p_i$

مرحله چهارم: محاسبه احتمال موفقیت p_i

تعداد کل یال‌های ممکن: تعداد کل زوج راس‌های ممکن: $C(n,2) = \frac{n(n-1)}{2}$

تخمین تعداد یال‌های مفید: فرض می‌کنیم در حالت i مولفه، یکی از مولفه‌ها بزرگ (با $n - i +$ 1راس) و بقیه کوچک (هر کدام با ۱ راس) هستند.

محاسبه دقیق:

- تعداد یال‌های مفید ≈ $(i-1) \times (n-i+1)$
- تعداد کل یال‌های ممکن = $n(n-1)/2$

احتمال موفقیت:

p_i:تعداد یال‌های مفید / تعداد کل یال‌های ممکن

$$p_i = (i-1)(n-i+1) / [n(n-1)/2] = 2(i-1)(n-i+1) / [n(n-1)]$$

برای n بزرگ: $p_i \approx 2(i-1)/n$

مرحله پنجم: محاسبه $E[X_i]$

$$E[X_i] = 1/p_i = n/[2(i-1)]$$

مرحله ششم: محاسبه امید ریاضی کل

$$E[X] = \Sigma(i = 2\ to\ n)\ E[X_i] = \Sigma(i = 2\ to\ n)\ n/[2(i-1)]$$

با تغییر متغیر $j = i - 1$: $E[X] = (n/2) \times \Sigma(j = 1\ to\ n-1)\ (1/j) = (n/2) \times H_(n-1)$

که H_k عدد هارمونیک k-ام است.

مرحله هفتم: نتیجه نهایی

فرمول دقیق: $E[X] = (n/2) \times H_(n-1)$

تقریب برای n بزرگ: چون $H_k \approx ln(k) + \gamma$ (γ ثابت اویلر):

$$E[X] \approx (n/2) \times ln(n-1) \approx (n\ ln\ n)/2$$

توضیحات تکمیلی:

۱. چرا از توزیع هندسی استفاده کردیم؟ در هر مرحله، یال‌ها به صورت تصادفی و مستقل انتخاب می‌شوند. احتمال اینکه یال انتخابی "مفید" باشد (یعنی دو مولفه مختلف را وصل کند) ثابت است. ما به دنبال اولین یال مفید هستیم، که دقیقاً تعریف توزیع هندسی است.

۲. توجیه تخمین تعداد یال‌های مفید: در عمل، اندازه مولفه‌ها متغیر است، اما برای سادگی محاسبات، بدترین حالت را در نظر گرفتیم که یک مولفه بزرگ و بقیه تک‌راسی باشند. این تخمین کران پایینی مناسبی ارائه می‌دهد.

نتیجه نهایی: امید ریاضی تعداد یال‌های مورد نیاز برای همبند شدن گراف برابر $(n\ ln\ n)/2$ است.

سوال ۶- لحظه وصال: وقتی گراف بالاخره همبند می‌شود

صورت سوال:

سوال ۶: فرصت طلایی: فقط یک بار حق انتخاب داری

در یک سایت حراجی، روال کار به این صورت است که:

۱. یک فروشنده، محصولی را برای فروش ارائه می‌دهد.

۲. خریداران به ترتیب (i) وارد شده و پیشنهاد خود (b_i) را ارائه می‌دهند.

۳. فروشنده به محض ورود یک پیشنهاد، تنها دو انتخاب دارد: بپذیرد یا رد کند. در صورت پذیرش، کالا فروش رفته و کار فروش تمام است و در صورت رد، آن پیشنهاد از سامانه حذف شده و پیشنهاد بعدی به فروشنده نشان داده می‌شود.

الگوریتمی طراحی کنید که به فروشنده امکان دهد با دانستن تعداد کل خریداران (n) و در نظر گرفتن پیشنهادهای ارائه شده، شانس قبولی برای فروش کالا به بالاترین قیمت را داشته باشد.

برای مثال فرض کنید فروشنده همیشه اولین انتخاب را قبول کند. در این صورت شانس او برای فروش به بالاترین قیمت $\frac{1}{n}$ خواهد بود.

الگوریتمی که شما طراحی می‌کنید، باید طوری باشد که مستقل از مقدار n، فروشنده کالای خود را با حداقل احتمال $\frac{1}{4}$ به بالاترین پیشنهاد بفروشد.

در این سوال، با مسئله‌ای مواجه هستیم که در نظریه تصمیم‌گیری به مسئله "انتخاب دبیر" یا "مسئله توقف بهینه" معروف است. فروشنده باید تصمیم بگیرد چه زمانی پیشنهاد را بپذیرد تا احتمال انتخاب بالاترین پیشنهاد حداکثر شود.

الگوریتم پیشنهادی

الگوریتم ما دو مرحله اصلی دارد:

مرحله ۱: مشاهده و کالیبراسیون

نقطه برشی k را تعیین می‌کنیم (k = n/2 انتخاب مناسبی است)

پیشنهادهای خریداران ۱ تا k را بدون پذیرش مشاهده می‌کنیم

بالاترین پیشنهاد مشاهده شده در این دوره را به عنوان معیار (Bk) ذخیره می‌کنیم

مرحله ۲: انتخاب

برای خریداران k+1 تا n:

هر پیشنهاد جدید bi را با Bk مقایسه می‌کنیم

اولین پیشنهادی که از Bk بالاتر باشد را می‌پذیریم و فرآیند تمام می‌شود

اگر تا خریدار آخر هیچ پیشنهادی از Bk بالاتر نبود، پیشنهاد آخر را می‌پذیریم

تحلیل احتمال موفقیت

برای اثبات اینکه احتمال موفقیت این الگوریتم حداقل ۱/٤ است، باید شرایط موفقیت را بررسی کنیم:

شرایط موفقیت

فرض کنید $Bmax$ بهترین پیشنهاد کل در جایگاه i-ام ($1 \leq i \leq n$) ارائه می‌شود. برای موفقیت الگوریتم:

۱. **شرط اول**: بهترین پیشنهاد نباید در دوره مشاهده باشد ($i > k$)

۲. **شرط دوم**: الگوریتم باید پیشنهاد i-ام را انتخاب کند، یعنی بالاترین پیشنهاد در k پیشنهاد اول باید در موقعیتی بین ۱ تا i-1 باشد

محاسبه احتمال موفقیت

احتمال موفقیت برابر است با مجموع احتمالات موفقیت برای تمام جایگاه‌های ممکن i:

$$P(success) = \sum_{i=k+1}^{n} P\,(\text{موفقیت به شرطی که } Bmax \text{ در جایگاه } i \text{ باشد})$$

اگر $i \leq k$ باشد، احتمال موفقیت صفر است. اگر $i > k$ باشد، احتمال موفقیت برابر است با احتمال اینکه $Bmax$ در جایگاه i باشد (n/۱) ضربدر احتمال اینکه بالاترین پیشنهاد بین k تای اول، در بین i-1 تای اول باشد ($k/(i-1)$):

$$P(success) = \sum_{i=k+1}^{n} \frac{1}{n} \cdot \frac{k}{i-1} = k/n \sum_{i=k+1}^{n} \frac{1}{i-1}$$

اثبات کران پایین برای $k = n/2$

با قرار دادن $k = n/2$ در فرمول بالا:

$$P(success) = \frac{\frac{n}{2}}{n} \sum_{i=\frac{n}{2}+1}^{n} \frac{1}{i-1} = 1/2 \sum_{j=n/2}^{n-1} 1/j$$

کران پایین برای این مجموع:

$$\sum_{j=\frac{n}{2}}^{n-1} \frac{1}{j} \geq \frac{\frac{n}{2}}{n} \cdot \frac{1}{n} = \frac{1}{2}$$

بنابراین:

$$P(success) \geq 1/2 \cdot 1/2 = 1/4$$

توضیحات تکمیلی

چرا این الگوریتم کار می‌کند؟

این الگوریتم تعادلی بین "یادگیری" و "بهره‌برداری" برقرار می‌کند. در مرحله اول، با مشاهده نیمی از پیشنهادها، معیاری برای ارزیابی پیشنهادهای بعدی به دست می‌آوریم. در مرحله دوم، از این معیار برای تصمیم‌گیری استفاده می‌کنیم.

دلیل انتخاب k = n/2

انتخاب k = n/2 بهینه است زیرا:

- اگر k خیلی کوچک باشد، معیار ما برای تصمیم‌گیری ضعیف خواهد بود
- اگر k خیلی بزرگ باشد، ممکن است بهترین پیشنهاد را در دوره مشاهده از دست بدهیم

مقایسه با انتخاب تصادفی

اگر فروشنده به صورت تصادفی یک پیشنهاد را انتخاب کند، احتمال انتخاب بهترین پیشنهاد فقط $۱/n$ است. الگوریتم ما با احتمال حداقل ۱/٤، بدون وابستگی به n، بهترین پیشنهاد را انتخاب می‌کند.

کاربردهای عملی

این الگوریتم در مسائل مختلف تصمیم‌گیری کاربرد دارد:

- استخدام کارمندان (مصاحبه با تعدادی کاندیدا برای تعیین معیار، سپس انتخاب اولین کاندیدای بهتر از معیار)
- خرید خانه یا ماشین
- سرمایه‌گذاری در بازارهای مالی

نتیجه‌گیری

با رد کردن $n/2$ پیشنهاد اول و سپس انتخاب اولین پیشنهادی که از بهترین پیشنهاد مشاهده شده بالاتر است، احتمال فروش کالا به بالاترین قیمت حداقل ۱/٤ یا ۲۵٪ تضمین می‌شود. این الگوریتم کران پایین قابل قبولی برای موفقیت، مستقل از مقدار n فراهم می‌کند.

((پایان))